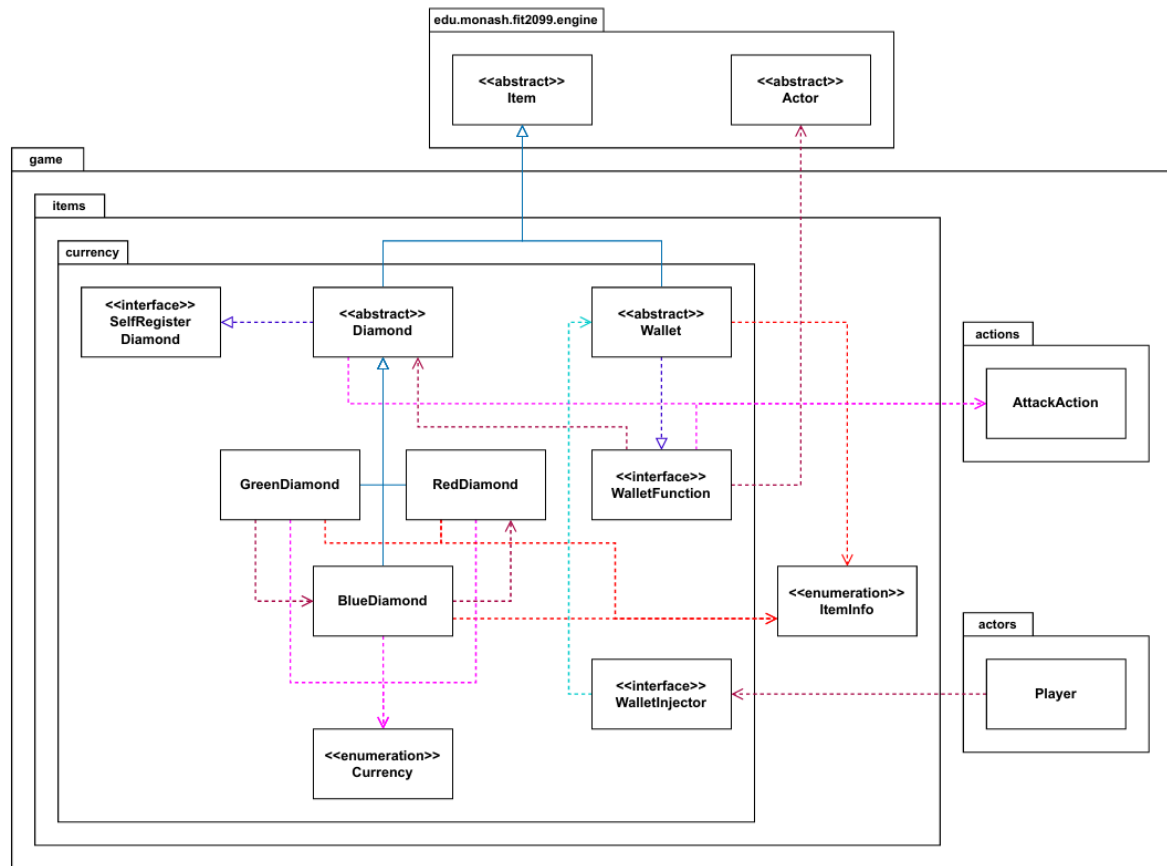


Design Rationale—Requirement 4: Diamond Currency System

Design Diagram



Design 1

1. Each time the specific diamond type (Green, Blue, Red) to drop when actors defeated an animal. This is implemented within the attack logic (e.g., AttackAction).
2. These diamond is stored directly in the actor's inventory. The combination logic (2 Green Diamonds can combine to a Blue Diamond) is manually handled through ad-hoc loops scattered inside the actor class.
3. No abstraction layer separates the collection and combination logic, so both are embedded inside gameplay logic.

Pros	Cons
Simple and explicit to follow.	High connascence of execution: The order of diamond creation and combination must be correct because any missing conditional leads to inconsistent state or crashes.
Fast to implement since all logic resides in one place.	Violates SRP and DRY: The same logic (diamond creation, combination, and value retrieval) is repeated across multiple gameplay classes.
Works for a fixed set of diamond types.	Strong coupling: Gameplay logic directly depends on specific diamond subclasses.
	Difficult to extend: Adding a new diamond tier requires editing multiple conditional statements in several classes.
	No scalability: Combination rules and diamond values are not centralized, making the system error-prone and hard to maintain.

Design 2

1. Create an abstract superclass Diamond that defines shared diamond behaviour.
2. Create concrete subclasses GreenDiamond, BlueDiamond, and RedDiamond, each responsible for their tier's value and next-tier mapping.
3. Implement a self-registration mechanism so each subclass automatically registers itself into the diamond registry upon class loading, eliminating manual linking.
4. Create a Wallet class implementing WalletFunction, responsible for storing, combining, and managing diamonds automatically when collected.
5. An injector interface provides a method to create wallets without depending on the concrete class.
6. An enum class defines constant diamond values in one centralized place.

Pros	Cons
Encapsulation: Diamond creation, registration, and combination are self-contained within the currency subsystem.	Slightly more setup code compared to manual instantiation.
Eliminates connascence of execution: Diamond registration and combination are automatic, so order no longer matters.	
Fully follow SOLID principles: - Single Responsibility Principle: Diamond handles tier logic; Wallet handles storage and combination; Currency initializes each diamond values. - Open Closed Principle and Future extensibility: New diamond types can easily extends Diamond superclass and registering automatically, without any changes of existing code. - Liskov Substitution Principle: All Diamond subclasses behave consistently and can replace each other where Diamond is expected. - Interface Segregation Principle: Functionality is segregated into small interfaces reducing unnecessary dependency exposure. - Dependency Inversion Principle: Game logic depends on abstractions (e.g., WalletInjector, WalletFunction) instead of concrete implementation.	
Low coupling and high cohesion: Wallets manage only diamond counts, Diamond handle only tier behaviour.	
Respects DRY: Combination logic and registration exist only once, in reusable form.	

Design 2 is more prefer because it significantly improves clarity, maintainability, and scalability. It internalizes the complex setup logic (registration and combination) into self-contained modules, removing connascence of execution and reducing human error during initialization. Not only that, the design strictly follow SOLID, DRY, and KISS principles by

clearly separating responsibilities of each class, ensuring the ease of expansion for future classes, and simplifying interactions between the actor, wallet, and item systems.

Existing classes and responsibilities

1. Abstract class
 - a. Diamond: Common diamond behaviour and registry.
2. Interfaces
 - a. SelfRegisterDiamond: Enables diamond classes to auto-register at runtime.
 - b. WalletFunction: Defines wallet operations (collect, remove, display balances).
 - c. WalletInjector: An injector to create an instance for promoting dependency inversion.
3. Concrete classes
 - a. GreenDiamond, BlueDiamond, RedDiamond: A type of diamond.
 - b. Wallet: A container to store, combine, and manage diamond; it responsible for tracking current balances and auto combination logic.
4. Enumeration
 - a. Currency: Centralize diamond values.

Classes related to and interacting with existing system

1. Animal defeat logic: In the AttackAction class, calls Diamond class method to spawn a random diamond.
2. Create Wallet instance via WalletInjector to the Actor inventory.
3. When animal is defeated by the actor, invokes the Wallet class method to collect diamond and internally handles automatic combinations of diamonds.
4. The wallet can display current balances using WalletFunction interface method.

Delivered functionality

1. When the actor defeats an animal, the system generates a random diamond drop.
2. When collecting diamonds, the Wallet automatically stores and combines diamonds based on predefined rules: 2 GreenDiamond -> 1 BlueDiamond, 2 BlueDiamond -> 1 RedDiamond.

3. Wallet maintains counts of each tier and displays readable balances.
4. Diamonds registration ensured all diamond types are known to the game at startup, avoiding missing-class errors.

Conclusion

The second design replaces the repetitive, fragile, and execution order dependent approach with a clean, modular architecture that isolates all currency related logic. Moreover, this design also reduces connascence of execution, enforces low coupling and high cohesion, as well as provides an easily extensible system for future diamond tiers or other collectible currencies. Therefore, the new design ensures stability, maintainability, and scalability, allowing seamless integration with existing game logic and minimal code duplication.